

web3Crypto

База знаний: web3-разработка и крипто-пет-проект

Экспорт — 2026-07-09

Содержание

1. index
2. Блокчейн-как-это-работает
3. Главная
4. Словарь-web3
5. Сравнение-ethers-viem-wagmi

Index — web3Crypto

Каталог страниц базы знаний. Domain: web3-разработка и крипто-пет-проект. Актуально: 2026-07-09, начало проекта.

Meta

- Главная — дорожная карта изучения: 5 этапов, технологии, идеи пет-проектов

Концепты

- Блокчейн — как это работает — основы: блоки, хеши, консенсус, газ, MetaMask, Etherscan
- Словарь терминов web3 — A-Z глоссарий: блокчейн, смарт-контракты, DeFi, NFT

Инструменты

- ethers vs viem vs wagmi — сравнение библиотек для фронтенда, рекомендации

Проект

- пока пусто — выбор пет-проекта на этапе 5

Блокчейн-как-это-работает

Блокчейн — как это работает

Объяснение для frontend-разработчика: без математики, на знакомых аналогиях.

Что такое блокчейн: две фразы

Блокчейн — это база данных, которую никто не может подделать. Данные хранятся не на одном сервере, а на тысячах компьютеров одновременно. Каждый новый «файл» (блок) ссылается на предыдущий — как git, но без force push.

Аналогия для фронтендера: представь Redux-стор, который хранится не в браузере, а в облаке. Каждый экшен навсегда остаётся в истории. Нельзя `reset()`, нельзя `REVERT_STATE`. Только `dispatch()`.

Как устроен блокчейн

Блоки



Каждый блок содержит: - **Хеш** этого блока (уникальный отпечаток) - **Хеш предыдущего блока** (связь в цепочку) - **Данные** (транзакции) - **Timestamp** — когда создан

Если изменить хоть байт в блоке #100, его хеш поменяется → блок #101 укажет на несуществующий хеш → цепочка порвётся. **Обнаруживается мгновенно.**

Хеш (криптографический отпечаток)

Хеш-функция берёт любые данные и выдаёт строку фиксированной длины:

```
sha256("hello") → 2cf24dba5fb0a30e26e83b2ac5b9e29e1b161e5c1fa7425e73043362938b9824
sha256("Hello") → 185f8db32271fe25f561a6fc938b2e264306ec304eda518007d1764826381969
```

Видишь: «h» → «H», а хеш другой полностью. Это свойство — лавинный эффект — делает блокчейн защищённым от подделок.

Консенсус: как все договариваются

Тысячи компьютеров (нод) хранят копию блокчейна. Как они договариваются, какая версия правильная?

Proof of Work (PoW, Bitcoin): реши математическую задачу → получи право создать блок. Тратит электричество.

Proof of Stake (PoS, Ethereum сейчас): положи ETH в залог → получи право создавать блоки. Если обманешь → потеряешь залог. Экологично, быстро.

PoS валидация блока:

1. Пользователи отправляют транзакции
2. Валидатор (случайно выбранный из залогов) собирает их в блок
3. Остальные валидаторы проверяют блок
4. Если всё ок → блок добавляется, валидатор получает награду

Ethereum — детальнее

Ethereum — это не просто «биткоин с программами». Это **мировой компьютер**: тысячи нод выполняют один и тот же код и приходят к одному результату.

Два типа аккаунтов

Тип	Кто контролирует	Пример
EOA (Externally Owned Account)	Приватный ключ	Твой кошелек MetaMask
Contract Account	Код смарт-контракта	Контракт токена USDC

Важно: **только EOA может инициировать транзакцию**. Контракт может выполнять код только когда его дёрнули извне.

Транзакция — что внутри

```
{
  "from": "0xТвойАдрес",      // отправитель
  "to": "0xАдресКонтракта",    // получатель (или 0x0 для деплоя)
  "value": "1000000000000000000", // 1 ETH в wei
  "data": "0xa9059cbb...",     // закодированный вызов функции
  "gasLimit": "21000",          // макс. gas на исполнение
  "maxFeePerGas": "50000000000", // макс. цена за gas (50 gwei)
  "nonce": 5                    // порядковый номер
}
```

Gas — это плата за вычисления. Каждая операция стоит определённое количество gas. Сложил два числа — 3 gas. Записал в storage — 20000 gas.

Зачем gas? Чтобы никто не заспамил сеть бесконечным циклом. Gas кончится → транзакция откатится.

EIP-1559 (текущий механизм gas): - `maxFeePerGas` — максимум, что ты готов заплатить за единицу gas - `maxPriorityFeePerGas` — чаевые валидатору (чем выше, тем быстрее включат) - Неиспользованный gas возвращается

Nonce — защита от повторной отправки

Каждый твой аккаунт имеет счётчик nonce. Отправил транзакцию → nonce++. Следующая транзакция должна иметь nonce на 1 больше.

Зачем: предотвращает повторную отправку одной и той же транзакции. Даже если злоумышленник перехватит подписанную транзакцию и попытается отправить её снова — нонсе уже не совпадёт, транзакцию отвергнут.

Практика: первое знакомство с сетью

1. Установи MetaMask

Расширение для Chrome/Brave: <https://metamask.io>

При создании кошелька тебе дадут 12 слов (seed-фраза). Это твой мастер-ключ. Запиши на бумаге. Не в облако. Не в заметки. Если потеряешь — потеряешь всё.

2. Переключись на тестовую сеть

В MetaMask: нажми на сеть (сверху) → «Показать тестовые сети» → выбери **Sepolia**.

Тестовые сети — это полные копии Ethereum, где ETH ничего не стоит. Можно ломать, терять, экспериментировать.

3. Получи тестовый ETH

1. Иди на <https://sepoliafaucet.com> (нужен аккаунт Alchemy, бесплатно)
2. Или <https://cloud.google.com/application/web3/faucet/ethereum/sepolia>
3. Вставь свой адрес (0x...) → получи 0.5 SepoliaETH

Адрес можно скопировать в MetaMask — клик на название аккаунта.

4. Отправь первую транзакцию

В MetaMask: «Отправить» → любой тестовый адрес (хоть свой же) → 0.01 ETH → подтвердить.

5. Посмотри на Etherscan

Etherscan — это «браузер блокчейна». Как DevTools для сети.

- Найди свою транзакцию на <https://sepolia.etherscan.io> (вставь свой адрес в поиск)
- Что ты увидишь: статус (Success), хеш транзакции, блок, gas, from, to

Поздравляю — ты только что написал в мировую базу данных, которая живёт на тысячах компьютеров.

Ключевые выводы первого этапа

1. Блокчейн = распределённая неизменяемая БД
2. Каждый блок ссылается на предыдущий через хеш → цепочка
3. Ethereum = «мировой компьютер», выполняющий код на всех нодах
4. Газ = плата за вычисления, защита от спама
5. MetaMask + Sepolia + Etherscan = твой инструментарий разработчика

Что дальше

- Смарт-контракты — введение — что такое смарт-контракт и как он работает
- Словарь терминов — если встретил незнакомое слово

Дорожная карта: web3 для frontend-разработчика

Зачем web3 фронтендеру

Web3 — это децентрализованные приложения (dApps), где бэкенд заменён смарт-контрактами в блокчейне. Для фронтендера это означает:

- **Новый тип бэкенда** — не REST API, а вызовы смарт-контрактов через ethers.js / wagmi
- **Новый тип состояния** — не Redux, а блокчейн (ончейн) + локальное (офчейн)
- **Новые UX-паттерны** — подтверждение транзакций, ожидание блоков, gas fees, wallet connect
- **Новый тип пользователя** — с криптокошельком (MetaMask, WalletConnect) вместо логина/пароля

Что входит в web3 (обзор)

[диаграмма]

План изучения (5 этапов)

Этап 1. Блокчейн-основы (1-2 недели)

Цель: понимать, как работает блокчейн на концептуальном уровне.

Что изучить: - [] Как устроен блокчейн: блоки, хеши, консенсус (PoW vs PoS) - [] Ethereum: аккаунты (EOA vs контракт), транзакции, gas, nonce - [] Кошельки: MetaMask, приватные ключи, seed-фразы, публичный адрес - [] Транзакции: что происходит при отправке ETH / вызове контракта - [] Etherscan: как читать транзакции и контракты

Практика: - [] Установить MetaMask, получить тестовый ETH из крана (Sepolia faucet) - [] Отправить тестовую транзакцию, посмотреть её на Etherscan

Этап 2. Смарт-контракты (Solidity) (2-3 недели)

Цель: написать и задеплоить простой смарт-контракт.

Что изучить: - [] Solidity basics: типы данных, функции, модификаторы, события - [] Хранение: storage vs memory vs calldata - [] Маппинги, массивы, структуры - [] Payable функции, transfer, send - [] ERC-20 (токены) — стандарт, как устроен - [] ERC-721 (NFT) — стандарт, mint, метаданные - [] Hardhat — среда разработки: компиляция, тесты, деплой - [] OpenZeppelin — библиотека проверенных контрактов

Практика: - [] Написать простой контракт-счётчик (increment/decrement + события) - [] Написать и задеплоить ERC-20 токен в тестовую сеть - [] Написать тесты на Hardhat (Chai + waffle)

Этап 3. Фронтенд dApps (2-4 недели)

Цель: подключить React-приложение к смарт-контракту.

Что изучить: - [] ethers.js v6 / viem — чтение и отправка транзакций - [] wagmi — React-хуки для web3 (useAccount, useContractRead, useContractWrite) - [] WalletConnect / RainbowKit — connect wallet UI - [] Обработка транзакций: pending → confirmed → receipt - [] Чтение событий (events/logs) из контракта - [] Subgraph (The Graph) — индексация ончейн-данных для быстрого чтения

Практика: - [] Простое dApp: connect wallet + прочитать баланс + отправить транзакцию - [] dApp к своему токenu: mint, transfer, просмотр баланса - [] Добавить события: слушать Transfer, показывать историю

Этап 4. DeFi и продвинутые темы (2-4 недели)

Цель: понимать, как устроены реальные web3-продукты.

- [] DEX (Uniswap): AMM, пулы ликвидности, свопы
- [] Lending (Aave): заём и кредитование, collateral
- [] Стейкинг, yield farming
- [] Layer 2: Arbitrum, Optimism, Base — зачем и как работают
- [] MEV, фронтраннинг, безопасность
- [] IPFS — децентрализованное хранение (для метаданных NFT)

Этап 5. Пет-проект

Цель: собрать полноценное dApp от контракта до фронтенда.

Идеи (ранжированы по сложности):

#	Идея	Сложность	Контракты	Фронтенд
1	Крипто-копилка — time-locked сбережения	..	Депозит с таймером, вывод после срока	Кошелёк + таймер + статистика
2	Лотерея — децентрализованный розыгрыш	...	Билеты, случайный победитель (Chainlink VRF)	Покупка билетов, история розыгрышей
3	Краудфандинг — сбор средств с soft/hard cap		Взносы, возврат если не собрано, claim	Кампании, прогресс-бар, статистика
4	NFT-маркетплейс — mint, buy, sell		ERC-721, листинг, торги	Галерея, mint-форма, профиль
5	DEX-агрегатор — лучший курс между DEX		Роутер, свапы через Uniswap/1inch API	Свап-форма, график цен

Технологический стек (рекомендуемый)

Слой	Технология	Зачем
Блокчейн	Ethereum (Sepolia testnet), потом Base/Arbitrum	Самый большой экосистема
Смарт-контракты	Solidity + Hardhat + OpenZeppelin	Индустриальный стандарт
Фронтенд	React + TypeScript + wagmi + viem	Привычный стек + web3
Wallet	RainbowKit / AppKit	Готовый UI для connect wallet

Слой	Технология	Зачем
RPC-провайдер	Alchemy / Infura	Доступ к блокчейну без своей ноды
Индексация	The Graph (Subgraph)	Быстрое чтение ончейн-данных

Первые шаги (сегодня)

1. Установить MetaMask, переключиться на Sepolia testnet
2. Получить тестовый ETH: <https://sepoliafaucet.com>
3. Прочитать первый концепт: [wiki/Блокчейн-как-это-работает](https://wiki.etherbase.io/Блокчейн-как-это-работает)

Связанное

- Словарь терминов web3
- Сравнение ethers.js, viem, wagmi

Словарь терминов web3

A

ABI (Application Binary Interface) — JSON-описание интерфейса смарт-контракта: какие функции есть, какие параметры принимают, что возвращают. Нужен фронтенду, чтобы знать, как вызывать контракт.

Account (аккаунт) — в Ethereum два типа: - **EOA (Externally Owned Account)** — обычный кошелек пользователя, контролируется приватным ключом - **Contract Account** — смарт-контракт, контролируется кодом

AMM (Automated Market Maker) — математическая модель обмена токенов без книги ордеров. $x \times y = k$. Используется в Uniswap.

B

Block — набор транзакций, подтверждённых сетью. Каждый блок ссылается на предыдущий (цепочка).

Blockchain — распределённый реестр, где данные хранятся в цепочке блоков. Нельзя изменить прошлое, не пересчитав всю цепочку.

C

Consensus (консенсус) — механизм, которым сеть договаривается о едином состоянии. Ethereum: PoS (Proof of Stake). Раньше: PoW (Proof of Work, как у Bitcoin).

D

dApp (Decentralized Application) — приложение, чья логика выполняется в смарт-контрактах, а не на сервере.

DeFi (Decentralized Finance) — финансовые сервисы на блокчейне: обмен, кредиты, стейкинг.

Deploy (деплой) — публикация смарт-контракта в блокчейн. После деплоя контракт получает адрес и его код нельзя изменить.

E

EOA — см. Account.

ERC-20 — стандарт взаимозаменяемых токенов (fungible tokens). Все токены одного контракта одинаковы (как доллары).

ERC-721 — стандарт невзаимозаменяемых токенов (NFT). Каждый токен уникален.

EVM (Ethereum Virtual Machine) — «компьютер», выполняющий код смарт-контрактов. Работает на всех нодах Ethereum.

Event (событие) — лог, который смарт-контракт записывает в блокчейн. Дешевле хранения, но нельзя прочитать из самого контракта (только извне).

F

Faucet (кран) — сервис, выдающий тестовые токены (ETH) для разработки в тестовых сетях.

G

Gas — плата за вычисления в Ethereum. Измеряется в gwei (1 gwei = 0.000000001 ETH). Каждая операция стоит определённое количество gas.

Gas limit — максимальное количество gas, которое ты готов потратить на транзакцию.

Gas price — цена за единицу gas. Чем выше, тем быстрее майнеры включают транзакцию в блок.

H

Hash (хеш) — односторонняя криптографическая функция. Одинаковый ввод → одинаковый хеш. По хешу нельзя восстановить исходные данные.

L

Layer 1 (L1) — основной блокчейн (Ethereum, Solana).

Layer 2 (L2) — решение поверх L1 для ускорения и удешевления транзакций (Arbitrum, Optimism, Base).

M

Mempool — «очередь» неподтверждённых транзакций, ожидающих включения в блок.

MetaMask — самый популярный браузерный криптокошелёк.

Mint — создание нового токена (обычно NFT).

N

Node (нода) — компьютер, участвующий в сети блокчейна. Хранит копию блокчейна и проверяет транзакции.

Nonce — порядковый номер транзакции от конкретного адреса. Предотвращает повторную отправку.

P

Private key (приватный ключ) — секретный ключ, дающий полный контроль над кошельком. Никому не показывать!

Public key / address (адрес) — публичный идентификатор кошелька (0х...). Можно показывать всем.

PoS (Proof of Stake) — механизм консенсуса: валидаторы ставят ETH как залог. Если обманут — теряют залог.

PoW (Proof of Work) — механизм консенсуса: майнеры решают сложные математические задачи. Энергозатратно (Bitcoin).

S

Seed phrase (сид-фраза) — 12/24 слов, из которых генерируются все приватные ключи кошелька. Хранить офлайн!

Smart contract (смарт-контракт) — программа, выполняющаяся на блокчейне. Код + состояние. После деплоя код неизменяем.

Solidity — основной язык программирования для Ethereum-смарт-контрактов. Похож на JavaScript.

Stablecoin (стейблкоин) — токен, привязанный к фиатной валюте (USDT, USDC = 1 USD).

T

Testnet (тестовая сеть) — копия Ethereum для разработки. Токены бесплатные, ошибки не страшны. Sepolia — текущая основная тестовая сеть.

Transaction (транзакция) — операция в блокчейне: перевод ETH, вызов функции контракта, деплой контракта.

W

Wallet (кошелёк) — программа/расширение для управления криптоактивами. Хранит приватные ключи и подписывает транзакции.

Wei — мельчайшая единица ETH. $1 \text{ ETH} = 10^{18} \text{ wei}$.

Сравнение-ethers-viem-wagmi

Сравнение ethers.js, viem, wagmi

Три основных инструмента для фронтенда, чтобы общаться с блокчейном. Какой и когда использовать?

ethers.js v6

Что это: низкоуровневая библиотека для взаимодействия с Ethereum. Прямой аналог web3.js, но современнее и легче.

Что умеет: - Подключаться к нодам через JSON-RPC - Читать данные из контрактов (`contract.balanceOf()`) - Отправлять транзакции (`contract.transfer()`) - Слушать события (`contract.on("Transfer", ...)`) - Работать с кошельками, подписывать сообщения

Плюсы: - Зрелая, самая популярная библиотека - Огромное количество примеров и tutorиалов - Полный контроль над всем

Минусы: - Тяжёлая (~500 KB бандл) - Не tree-shakeable - API местами неинтуитивно (особенно BigNumber в v5, в v6 улучшили)

Когда использовать: если нужен полный контроль и не пугает большой бандл.

viem

Что это: современная альтернатива ethers.js от создателей wagmi.

Что умеет: то же самое, что ethers.js, но:

Плюсы: - Лёгкая (~50 KB, tree-shakeable) - TypeScript-first (типы из коробки, без доп. пакетов) - Современное API: compose-able, предсказуемое - Быстрее развивается

Минусы: - Меньше tutorиалов и примеров (но быстро догоняет) - API отличается от ethers.js — нужно переучиваться

Когда использовать: новые проекты 2024+. Рекомендуемый выбор.

wagmi

Что это: React-хуки поверх viem (работает с viem v2, раньше работала с ethers.js).

Что умеет: - `useAccount()` — подключён ли кошелёк, какой адрес - `useConnect()` — диалог «подключить MetaMask» - `useReadContract()` — прочитать данные из контракта - `useWriteContract()` — отправить транзакцию - `useWaitForTransactionReceipt()` — ждать подтверждения - `useWatchContractEvent()` — слушать события

Плюсы: - Декларативный React-подход — меньше бойлерплейта - Автоматический refetch при смене аккаунта/сети - Кэширование, инвалидация запросов - Интеграция с RainbowKit (UI для connect wallet)

Минусы: - Абстракция — сложнее понять, что под капотом - Зависимость от React

Когда использовать: всегда, если пишем на React. wagmi + viem = золотой стандарт 2024+.

Как они соотносятся

Уровень абстракции:

```
wagmi      ← React-хуки (самый высокий уровень)
  ↓
viem        ← низкоуровневые вызовы (легковесный)
  ↓
ethers.js   ← низкоуровневые вызовы (тяжёлый)
```

Иерархия:

```
wagmi использует viem
viem — самостоятельная библиотека
ethers.js — самостоятельная библиотека
```

Рекомендация для пет-проекта

React + TypeScript

```
+ wagmi      (хуки: connect, read, write)
+ viem        (идет внутри wagmi, редко напрямую)
+ RainbowKit (UI для кнопки "Connect Wallet")
```

Не рекомендую ethers.js для новых проектов — viem легче, быстрее, и wagmi перешёл на viem.

Минимальный пример (wagmi + viem)

```
import { useAccount, useReadContract, useWriteContract } from 'wagmi';

function TokenBalance() {
  const { address } = useAccount();

  const { data: balance } = useReadContract({
    address: '0x...', // адрес токена
    abi: erc20Abi,
    functionName: 'balanceOf',
    args: [address],
  });

  const { writeContract } = useWriteContract();

  return (
    <div>
      <p>Баланс: {balance?.toString()}</p>
      <button onClick={() => writeContract({
        address: '0x...',
        abi: erc20Abi,
        functionName: 'transfer',
        args: ['0xRecipient...', 100n],
      })}>
        Отправить 100 токенов
      </button>
    </div>
  );
}
```

```
</div>  
);  
}
```